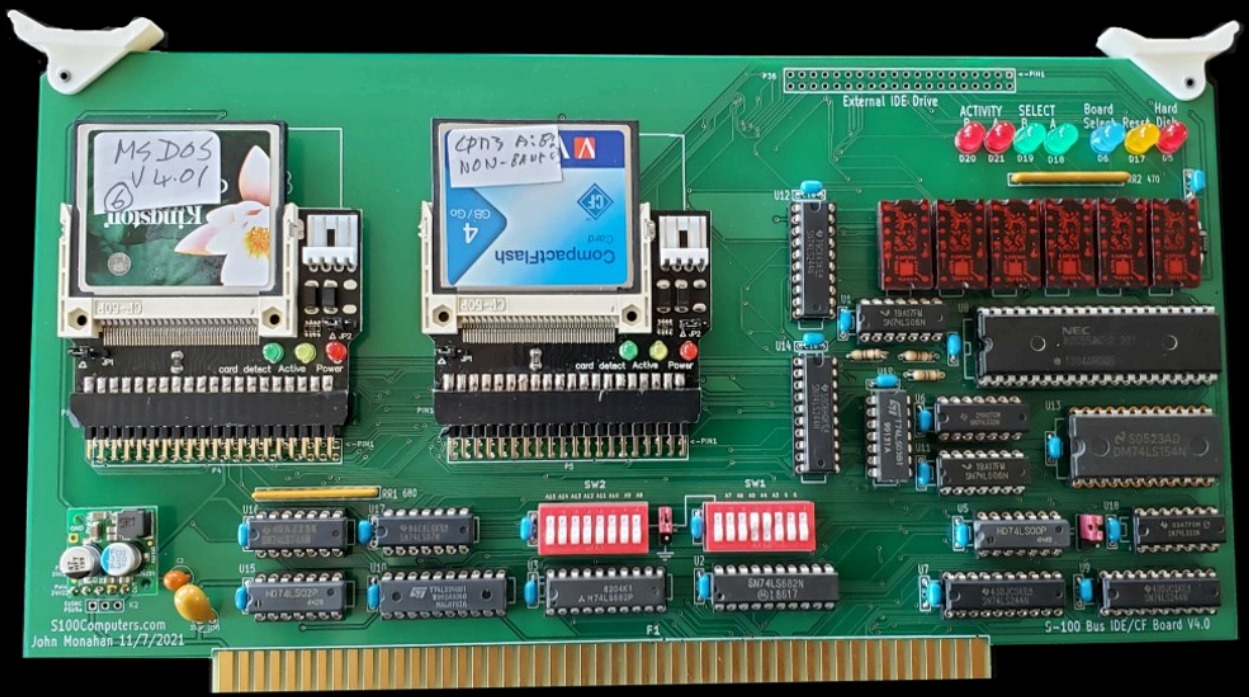
S100 Computers

A Web Site For S-100 Bus Computer Owners

An IDE Drive Interface S100 Bus Board



Dual CF-Card/Hard Disk IDE S100 Bus Board (V3)



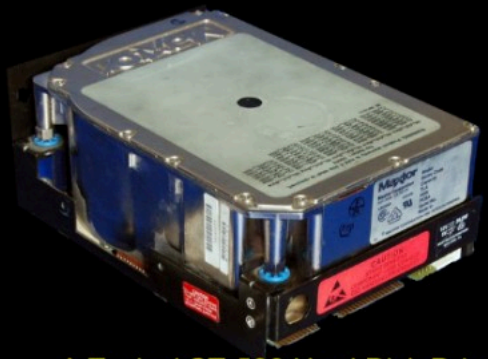
V4 Dual CF-Card/Hard Disk IDE Board

History of the IDE Interface.

Most later S-100 users have some kind of hard disk system to store and retrieve data. In the early 1980's the so called "Winchester" ST-506 drives were by far the most commonly used drives.

The ST-506 was the first 5.25 inch hard disk drive. Introduced in 1980 by Seagate Technology (then Shugart Technology), it stored up to 5 megabytes after formatting. With the ST-506 interface, the drive was connected to a controller card with two cables and power. The drives were "dumb", because the control card had to translate requests for a particular track and sector from the host system into a sequence of head positioning commands, then read the signal from the drive head and recovered the data from it.

A number of other companies quickly introduced drives using the same connectors and signals, creating a ST-506-based hard drive standard. IBM chose to use it, acquiring adapter cards for the PC/XT from Xebec and for the PC/AT, from Western Digital. As a consequence of IBM's endorsement, most of the drives in the 1980s were ST-506-based. However the complexity of the controller and cabling soon led to newer solutions like SCSI, and later, IDE.



A Typical ST-506 Hard Disk Drive



A Typical IDE Hard Disk Drive

The IDE drive interface itself has a long history of incremental technical drive interface developments over the years. IDE evolved initially from Western Digitals hard drive interface used in the above IBM-AT. The central idea of the IDE interface was to place the drive controller, data separator etc. on the drive itself. This was a major improvement over the earlier ST-506 interfaces. It allowed for more drive construction possibilities, better standardization, better reliability and a more flexible connection to the computer itself. There were a number of improvements and name changes occurred over the years such as ATA/ATAPI. After the market introduction of Serial ATA in 2003, IDE/ATA interfaces were retroactively renamed *Parallel ATA*. I shall still use the term IDE here.

There are now actually two types of IDE connectors the "standard' 40 pin connector found on all IBM-PC type motherboards and a newer 44 pin connector used mainly in laptops for small laptop drives. The extra 4 pins are used to bring power (+5V) to the drive. Here is a picture of a standard IDE connector:-



Introducing a Prototype S-100 Board to Interface with IDE Drives.

In the early 80's I too, used an ST506 hard disk drive. My initial drive was a Shugart 5Mg drive. Later I moved to a larger drive with both CPM and DOS partitions. The S-100 controller board I use is the Xcomp dual S-100 boards. I will write-up a description of this board and describe the software at a later stage.

Here I wish to describe a S-100 prototype board I constructed that allows one to interface the S-100 bus with a standard IDE drive interface. This increases tremendously the range of drives you can use to run CPM, CPM86 and MS-DOS with on the S-100 bus. Since the typical capacity of a CPM system is up to 8MG capacity, almost any old IDE drive is suitable.

Of particular usefulness is the ability to use IDE to Compact Card Flash memory cards with this board. This allows you to have a modern solid state drive on your S-100 system booting CPM silently and almost instantly. Here is the actual pinouts of a CF drive. There are many "IDE adaptors" available for a few $ that allow you to read and write to a CF card over an IDE connection. We will use them here.

The basic concept is to interface to the IDE drive(s) using a simple 8255 parallel IO chip. This was one of the workhorse parallel port LSI chips of the 70's. It contains 3 parallel ports which can be input, output, or bi-directional. The chip takes up only four I/O ports, one for control, three for the data. Programming the 3255 is a little tricky because it is an extremely powerful chip. There are many articles in the literature describing how to do this. The best is a book by Paul F. Goldsbrough "Microcomputing Interfacing with the 8255 PPI Chip", Howard Sams, 1979.

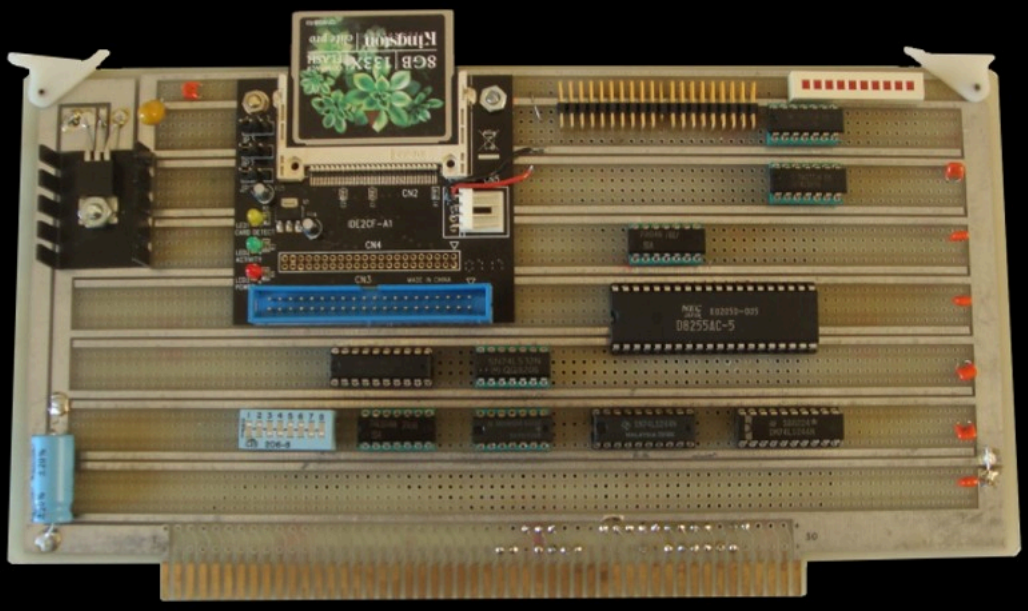
There was a pivotal article for interfacing an IDE drive written by Peter Fassee in 1998 where he interfaced an IDE drive using a 8255 with a 63B03 CPU. It can be seen here. This was further elaborated upon by a number of others over the years to work with other CPU's, for example an 8051 CPU, see here, (Both articles can be

located on the great web site for such things at www.pjrc.com). However I have not seen a layout for the S-100 bus with a Z80 CPU. That is what we have here.

What is most striking about the IDE interface when we compare it to the ST506 interface is how simple the software and hardware is. With the exception of the actual sector read and write commands of raw data, all commands are 8 bits. You simply setup the drive controller for a particular head, track and sector number. Then pulse a read or write signal and the drive delivers 512 bytes of data. For the real die hard's, the IDE specs can be seen here. The only complication is as Peter Fassee pointed out the switching of I/O modes in the 8255. In order to prevent glitching of signals, some lines on port C of the 8255 need to be inverted. See the above articles for more information.

The prototype board layout was fairly straightforward and can be seen below. Since there is plenty of room on the board I like to use plenty of diagnostic LED's to see what is going on. A full IEEE-696 addressing scheme of 24 address lines and full 16 bit (or just 8 bit) port addressing will be added to the final board (see below).

Here is a picture of the prototype board:-



IDE Interface S-100 Board With Compact Flash Card

This was my test system to debug the software. It used only 8 bit IO address decoding. The board has a standard IDE 40 pin connector at the top right hand side to hook-up to an IDE hard disk via a standard drive cable. Also on the board is connection to an IDE to CF flash card "adaptor". Today these "adaptors" are numerous, some costing as little as $8. I soldered the above card directly to the board using the adaptor cards protruding pins at the back of the CF IDE socket. Other cards have a male IDE pins that can attach to the board IDE pins if they are right angled. The LED readout in the above diagram is such that when everything is functioning normally they should all be lit (for the master drive).

Diagnostic Software

The board can be tested with a short diagnostic program (MYIDE.ASM) I wrote which can be seen here
The MYIDE.ASM file can be downloaded here:- MYIDE (V2-9).zip

Here is how the program looks when it first comes up.

```
IDE Disk Drive Test Program 3/14/2011 (V2.5) (Using CPM3 BIOS Routines)
CPM Track,Sectors --> LBA mode
Model: CF CARD 4GB
S/N:       5B0A06
Rev:   Ver3
Cylinders: 1E41, Heads: 0010, Sectors: 003F

                MAIN MENU
(L) Set LBA value  (R) Read Sector to Buffer (W) Write Buffer to Sector
(D) Display OFF    (S) Sequental Sec Read    (F) Format Disk
(V) Read N Sectors (X) Write N Sectors       (B) Backup disk
(G) Restore Backup (I) Next Sector           (J) Previous Sector
(U) Power Up       (N) Power Down            (C) Boot CPM
(ESC) Quit
Current settings:- CPM TRK = 0000 CPM SEC = 0000  (LBA = 00000001)

Please enter command >_
```

First a few comments on the code.

You must set the equates for the 4 ports of the 8255 LSI I/O chip. They must be contiguous. I use ports 30H-33H. Next you need to set the MAXSEC equate to the maximum number of sectors/track for your drive. This is typically written on the drive label itself. Since CPM systems will typically only utilize a fraction of a modern IDE drive if in doubt use a low number of sectors/track.

The next issue is how are you going to communicate with the keyboard/CRT/LCD display. If the equate for CPM=TRUE, communications will be via standard CPM/BDOS calls. However in cases where you do not yet have CPM up and running, if the CPM equate is set to FALSE, output will go to directly your console ports. These need to be set to their correct values. The code is very simple and straightforward. Its menu driven.

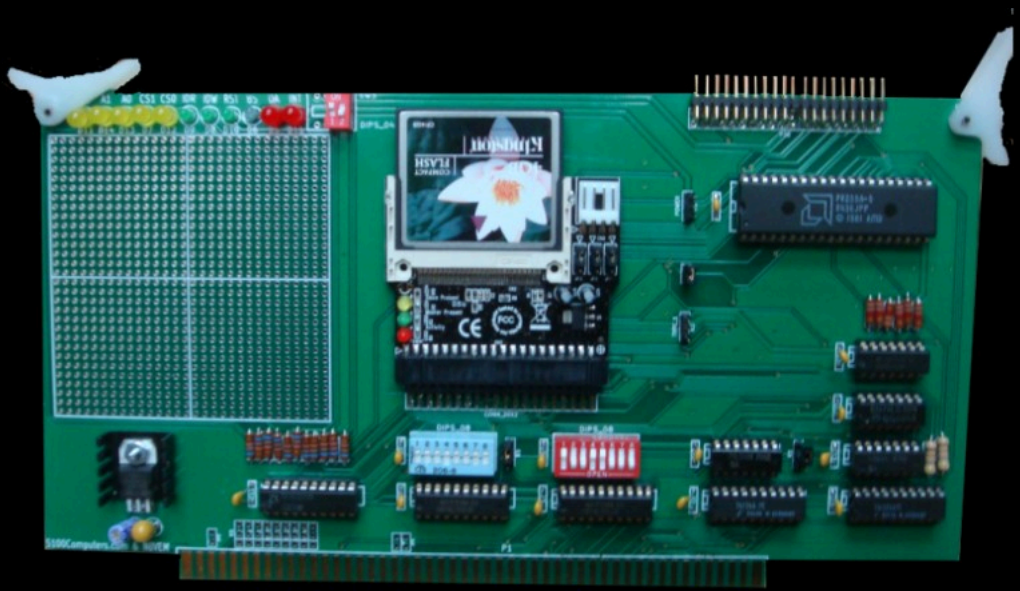
The program on starting reads the disk parameters and displays them. It then presents a short list of menu items.

Only the (L)BA menu item perhaps needs explaining. In the early days of IDE drives, sectors were defined in terms of tracks, sectors and heads. Sectors were a 16 bit number, tracks were a 32 bit number, and heads were a 4 bit number. This lead to incompatibility amongst drives with different numbers of sectors/track, or heads /drive. Software "drivers" had to be tailored to each drive. The MYIDE program can easily be modified to talk to the disk in this format. However these drives are really no longer used. Instead a "Logical Block Addressing" or LBA mode was developed. Here we address all sectors on the drive as 0 to the maximum number of sectors on the drive as one 24 bit number. We let the drive electronics figure out the actual track, sector and head. Unfortunately as we shall see later CPM still thinks in terms of tracks and sectors. So we must convert our CPM type sector/track request to a LBA sector request for the drive.

With the **"L"** command you enter a two hex digit sector number, a 2 digit hex high track number and then a 2 digit Hex low track number (tracks are 16 bits).For example 00,02,00 would give the first sector on track 2 (in CPM terminology) of the drive. The actual LBA number for my drive turns out to be 0000201H. You then hit **"R"** to read that sector. Then **"D"** will toggle a detailed display of the sector contents (@ 3000H in RAM) in hex. The **"S"** command will sequentially read one sector after the next continuously over the whole disk. A good way to be sure your hardware is sound.

The **"W"** command writes 512 bytes of whatever is in RAM at 3000H to a specified sector. Use with care! The **"N"** stops the drive spinning, the **"U"** command starts it up again. The **"F"** command formats each sector with "E5's". This is necessary to have an empty directory the first time you use the disk with CPM. MS-DOS/Windows does not do this the same way. Finally while you probably could write a format program to format these drives. I find it easier to pop them into a PC, Format them under windows in FAT32 mode and use them directly in my system. To avoid CPM's directory getting confused, I have a sector write command that writes 0E5's in sectors under the (F)format command in MYIDE.COM . You only need the first few tracks of course. To be on the safe side I always erase *.* with a new drive with CP/M anyway. The above of course is a one time only event. After that the drive can be treated like any other CPM drive. I have not used this board with MS DOS yet.

The **"V"** and **"X"** command allow you to rapidly read/write a block of sectors. This with care can be used to write/copy CPM system images to/from the system tracks. The **"B"** and **"G"** commands allow you to backup/restore the complete CPM disk image (8MG) to another location on the drive/CF card. Finally the **"C"** allows you to boot the CPMLDR file on the system tracks of the disk (if it is present). Some block moves in the code there since the CPMLDR must reside at 100H in RAM!



S-100 IDE Board with CF card adaptor

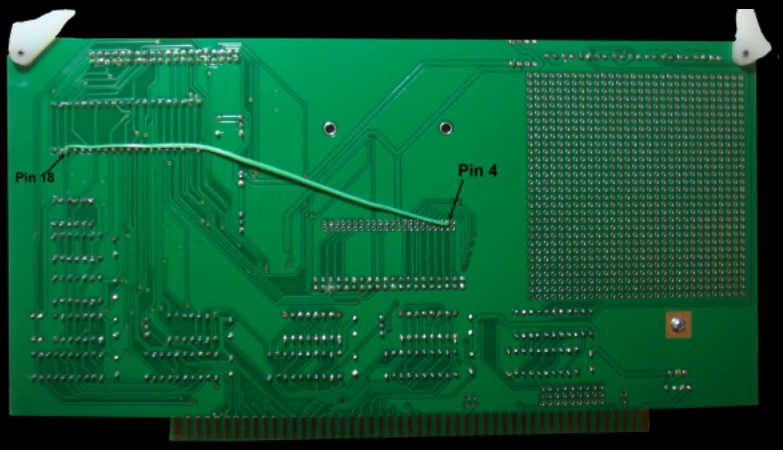
Here is a detailed schematic of the board.

Help Building The Board

Should you decide to build a board like this yourself, here are some useful tips and step by step instructions in doing so.

First install the voltage regulator and power supply capacitors. Make sure the capacitor on the +8 volts side of the regulator is rated for at least 15 volts. Install all the sockets and filter capacitors, resistors and LED's. For the LED's make sure the +5 connection (the longer LED lead) is orientated correctly before you solder it in. Then place the board in a system with an extender card and check the is 5 volts on each and every 20 pin and 14 pin socket (pins 20 & 14). Check pin 26 of the 8255 socket also is +5 volts. Carefully inspect each solder joint and make sure it is correct. There is a great ground plain on this board, however this causes the ground pin pods for each socket have heat conducted away well when soldering. I sometimes find it hard to get a good joint. I had one case where the board LED's worked fine only if the board was flexed. It later turned out to be a bad ground pin solder joint I did.

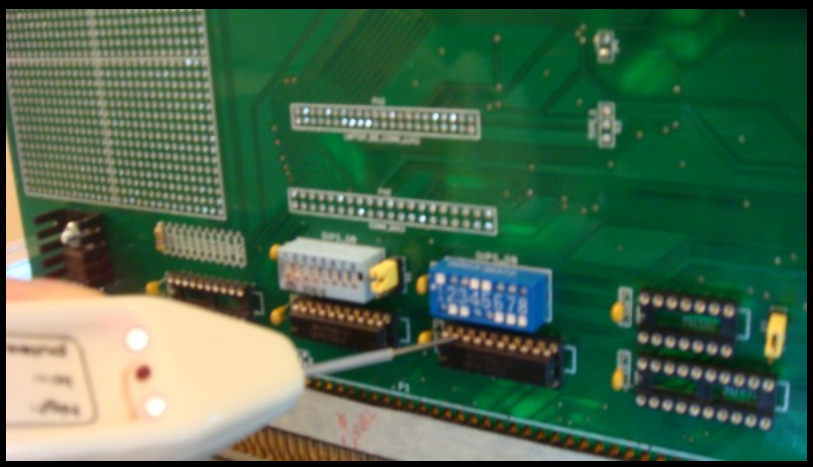
Unfortunately there was a slight error in the board's manufacture that requires a single jumper wire be connected from pin 18 of the 8255 (U8) to pin 4 of the connector P42. The picture below shows the back of the board with this long wire jumper. Make sure you use the correct P42 pin. It's the second pin in top row of pins.



Next install the two switches and only U2 and U3. We now will check port addressing. The board utilizes 74LS682's for port addressing. If your are unfamiliar with this technique click here.

Let's assume you will be using the 4 port block 30H to 33H. This is what I have the diagnostic software (MYIDE.ASM) port defaults. We will set the 16 bit I/O address range to 00xxH by jumpering K5 to 1-2 (the top two positions). Set switch SW2 positions 1-8 to closed. Set switch SW1 to 30-34H by having the switches (left to right):-
Open, closed, closed, open, open, closed, closed, closed.

Now put the card in your S-100 extender card and start the computer.
With your monitor at 0H in RAM enter:-
DB, 30, C3, 00, 00
Jump to 0H. With a probe you should have pin 19 of U2 pulse continuously. Do not go further until you see this.



Next add all the other IC's except the 8255, U7 and U9. Note, U4 MUST be a 74H04 or 74S04 (see below). The board will not work with IDE/CF cards with a 74LS04. (It seems to be OK for slower Hard Disks with a 74LS04 chip).

Repeat the above software test. Check that pin 1 & 19 of empty socket of U9 for a pulse. It should not pulse for any other value except 30H, 31H, 32H, & 33H. This insures we are able to read the data port correctly. Now the write to ports test.

Same software as above except we have:-
D3, 30, C3, 00, 00

Check for a pulse on pins 1 & 19 of empty socket U7. Again it should not pulse for any other value except 30H, 31H, 32H, & 33H
Only if you get this to work go further.

Insert the two 74LS244's (U7 & U9) and the 8255.

We will now checkout the 8255. A complex chips with many configuration possibilities/modes. Fortunately for testing we need only the simplest mode where all 3 ports (A, B & C) are set as simple outputs. To do this from your monitor send to its control port (33H) the byte 80H. With this, port A (30H), port B (31H) and port C (32H) are configured for output ports.

With the LED's on the board you can easily monitor port A.
Output to port C (32H), 0H. All the appropriate LED's should come on. Then output 01, 02, 04, 08, 10H, 20H, 40H and 80H. The appropriate LED should go off (see the above schematic).

Next we need to check port A (30H). Output to port 30H the value 0H. All the appropriate 8255 pins (4..1, 40..37) should be LOW. Then one at a time output a bit to this port. Starting at bit 0 send 01H to port 30H. See that pin 4 of the 8255 goes high. Continue all the way up to bit 7, pin 37 of the 8255.

Now do the same for the 8255 Port B (31H). First check the pins (18-25). You will need a copy of the boards schematic. See here.
Output to port 31H the value 0H. Pins 18-25 of the 8255 should be low. Output the single bits pattern above and check each appropriate 8255 pin goes high. Only if all of the above checks out should you go further.

Next we will check the actual IDE socket pins. Setup the 8255 to all ports output as we did before (Output 80H to port 33H) then output to port 31H the value 0H. ALL appropriate pins, 4,6…18 on any IDE socket should be 0. Then output a 1 to port 31H. ONLY pin 4 on each IDE socket should be high. Same for all the other bits. Only when you are sure that you can reliably output the correct bit pattern from port 31H of the 8255 to any of the IDE sockets should you go further.

The boards are extremely well made so there is little chance of partial etching. Any stuck bits are probably an error on your part.

With the above completed, you are ready to fire things up with an IDE drive. I have used the board with a number of drives and CF cards. I find it is easiest to get things going with an old IDE hard drive. Be sure you plug you plug the IDE cable in correctly. From the front of the board (V regulators on left) the top IDE socket has pin 1 on the top row at the very right back row of pins.

Download my "MYIDE.ASM" program to check things out. Run the program under CPM. The drive should come on and display the drive configuration parameters. See the picture above. The drive I used had 2082 cylinders, 16 heads and 63 sectors/track. You then need to read the sectors/track info you got and if required reassemble the program for your drive with the appropriate equates set in the software.

As I said, I have two of these new boards running here (on two systems). They appear very reliable. I have extensively slopped large files to/from memory disks and down to slow floppies. Never a problem. It's so nice to instantly have CPM3 boot up!

The only unusual thing I have found is the requirement of U4 being a fast 74H04 or 74S04 when using CF disks. It's an absolute requirement for the cards I have. Not so for regular hard disks. I have not really determined why this is the case.

You can "Hot Swap" the CF cards in a running CPM system if you first turn off power to the IDE adaptor/CF board (SW3), remove the CF card. Insert the new card and THEN turn the power back on with switch SW3. With CPM you need to warm boot with a ^C. Clearly there must be a system on the new CF card or you must do the swap where another drive is the current system drive.

Software

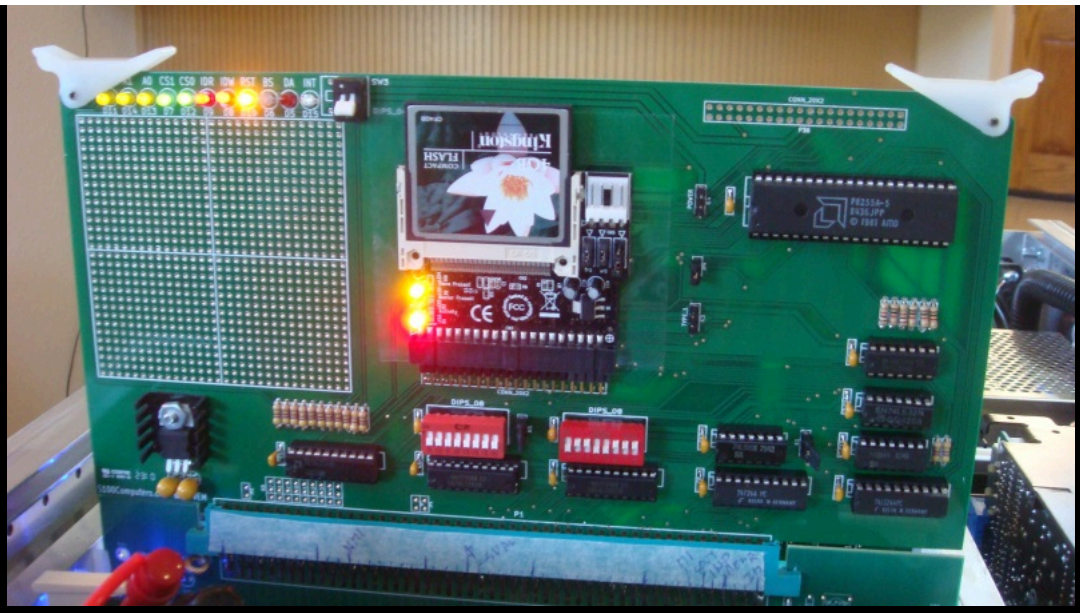
Installing CPM3 On The IDE Disk/Card

In order for this IDE card to be of any practical use we need to install CPM (and later, other disk operating systems) on drives interfacing to it. We could start with CPM version 2.2, but it is actually easier to write a BIOS for CPM3 because CPM3 takes care of buffer sizes greater than 128 bytes internally (all IDE drive sectors are 512 byte sectors) and is in general is a more practical and expandable system.

Before writing a CPM3 BIOS for this board please read the write-up I have in the software section of this site about setting up a CPM3 system on a floppy disk system. Please read here first. Then read here for details as to how to install CPM3 on a drive/CF card using this board.

Update on Version 2 of the S-100 IDE Board.

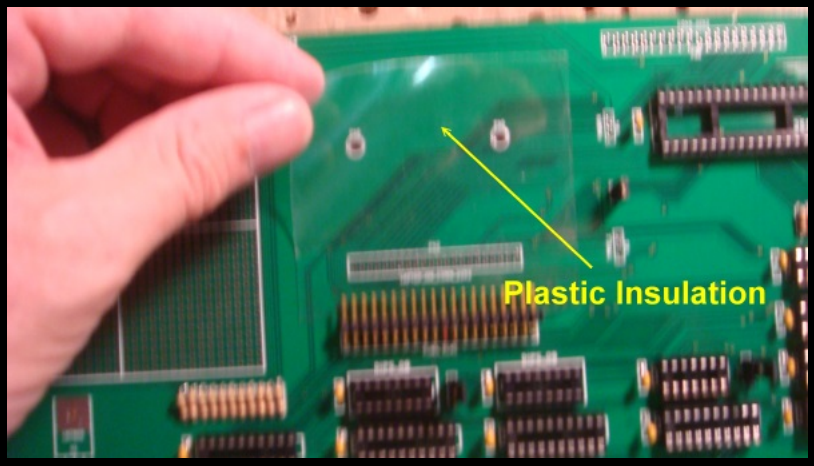
There was a surprising demand for this S-100 IDE board. We have now completed a second batch of boards. These boards are identical to the first batch except the pin 18 to pin 4 dropped trace (described above) has been corrected. Also on the second batch of boards the "Int" LED does not come on unless the IDE drive request it on its IDE INT line. Here is a picture of the batch 2 board.



The schematic for the board can be obtained here, and the board layout here.

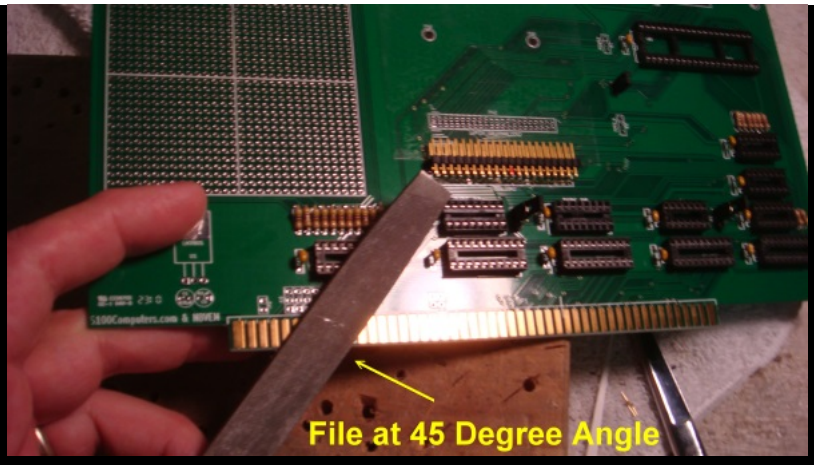
Two things I forgot to mention earlier when assembling these boards.

First it is important that the back pins/solder connections of the IDE-Flash Card adaptor do not come in contact with any of the traces on the actual S-100 board. To be on the safe side I add a layer of adhesive transparent plastic. See this picture:-



This insures that there will be no connection between the two.

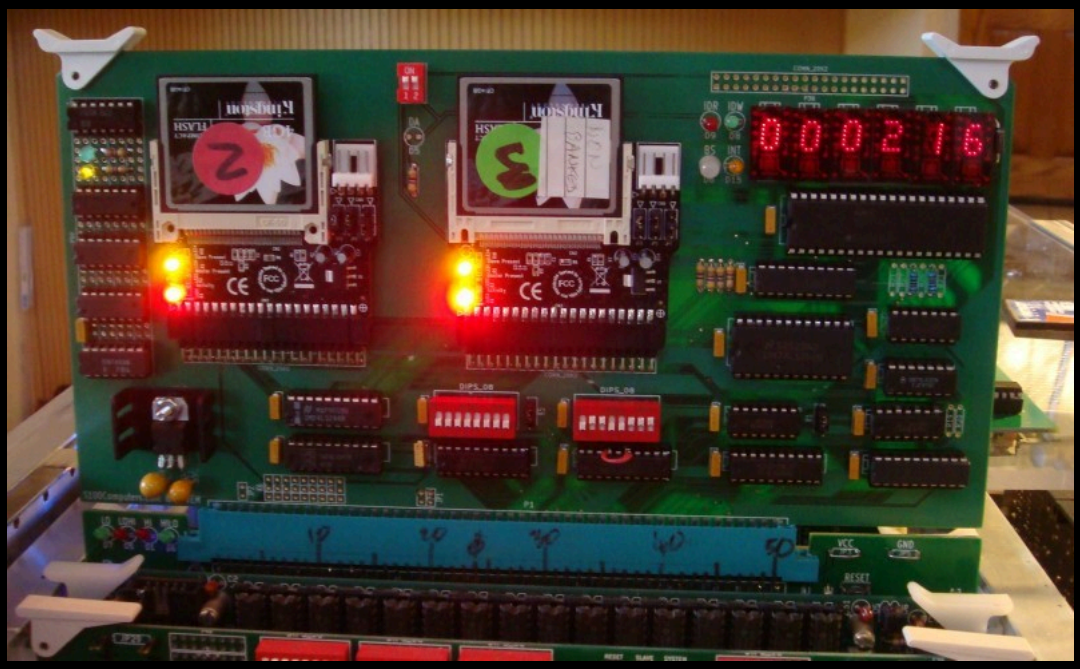
Second, these S-100 boards are of excellent quality and perfectly cut. However the bottom edge where the S-100 connections are is quite square sometimes making it difficult to fit the card into the bus edge connector. I find it is best to lightly run a file along the bottom front and back of the board to take off the edge as shown here:-



Please take extreme care not to damage the gold plated connections. The file should be at a 45 degree angle. Two or three sweeps on both sides is all that is needed.

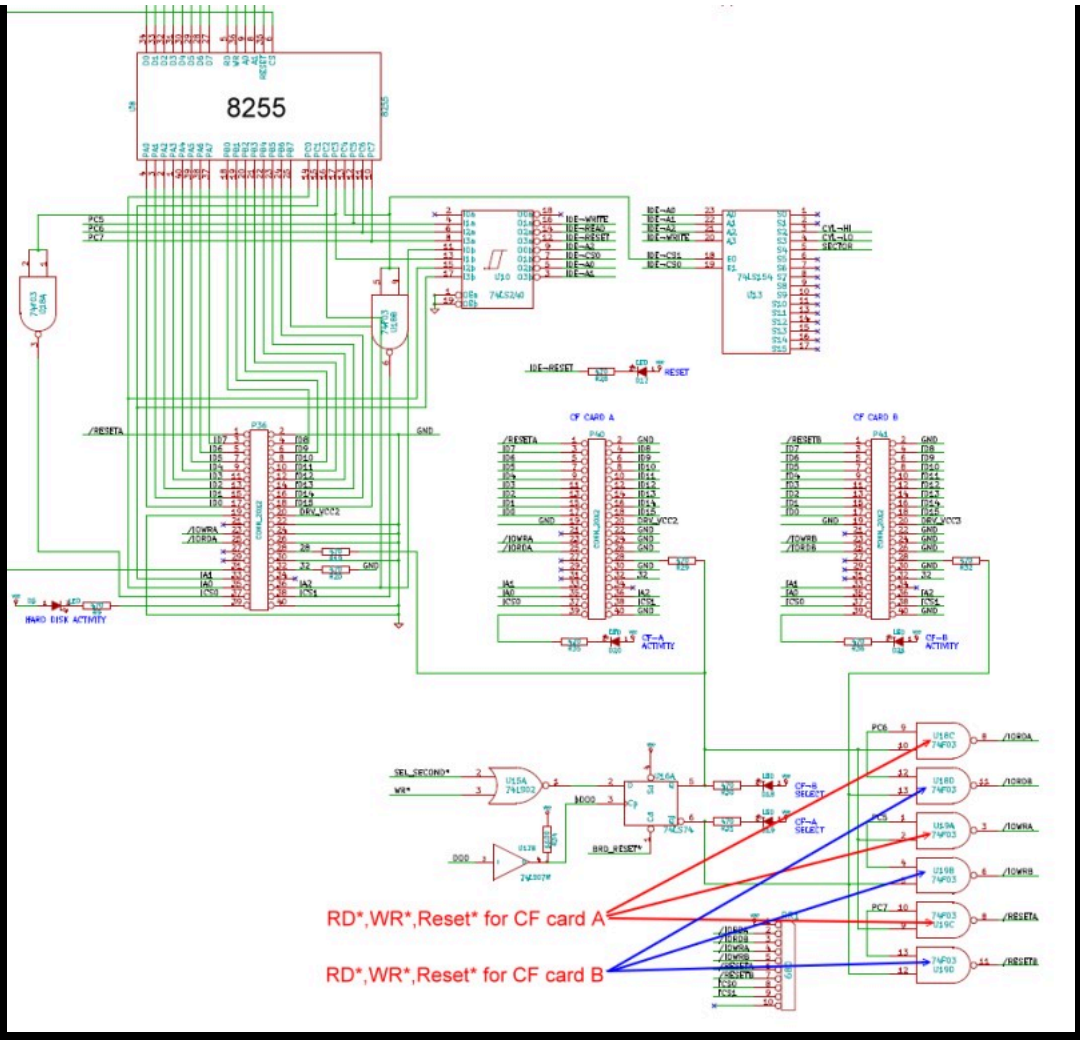
A Third Production S-100 Board.

Realizing that a number of people might want to utilize a board like this we have now completed a prototype of a **THIRD** (V2) IDE board. The previous two runs have be used up.



This board is very similar to the previous boards except that now it has two IDE slots for CF cards instead of one. This allows you to use two separate CF cards in your system. This is very useful for "disk/card" copying and backing up your software. The board also has an LED HEX display that shows the current sector being read or written to. From a software prospective it is identical to the previous boards. After a hardware reset the board comes up with the first drive/CF card active with the same software drivers as the V1 board. The second drive/CF card is set as the current active drive by outputting a bit 0 to the board address port +1. This way it is very easy to modify your CPM BIOS to address two drives instead of one.

It turned out to be more difficult than I expected to have two CF cards sharing the same IDE "bus". Normally this bus has a *master* and a *slave* drive. However I could not get both drives to initialize properly. This probably has to do with the fact that the master drive first initializes and only then can one initialize the slave drive. On the web there are references to "interference" in getting two CF drive systems working on the same IDE bus. In the end I took the simplest approach and setup the system where each drive (thinks) it is a master. This involves sending separate RD*, WR* and Reset* signals to each drive. A simple 7474 flip flop determines the "active" drive. Upon reset the first drive/CF card is always the current drive.
Here is the core part of the schematic.



You will also notice I did away with the 74H04 IC. I now understand why that IC was required in the V1 boards. It turns out the specs for certain signals on an IDE interface require a high signal of 4 volts (CMOS levels). 74LS04 signals could not get to these levels. Presumably the 74H04 signals did. The spec. requirements are easily met using open collector outputs. That is why I use 74F03's (actually even 74LS03's are fine) with pull-up resistors in the V2 board. In a 10Mhz Z80/CPM+ system the board works rock solid with many CF cards (Kingston, Maxell, Patriot etc.). However for some strange reason I could not get it to work with a "Ultra SanDisk 2GB card". I have not had time to check into this.

Finally, I cannot stress enough the importance of having a clean reset signal to these CF cards. I have not figured out the minimum length of a reset pulse, but here is the code I use for a Z80 running at 10MHz with 2 two I/O wait states. This gives a 60 uS pulse on pin one of the IDE connector and seem to work fine with 5 different CF cards I have.

```
INITILIZE_IDE_BOARD:              ;Drive Select in [A]. Note leaves selected drive as [A]
        LD      A,RDcfg8255       ;Config 8255 chip (10010010B), read mode on return
        OUT     (IDECtrl),A       ;Config 8255 chip, READ mode
        LD      A,IDEreset        ;IDEreset EQU 80H
        OUT     (IDECport),A
        LD      C,IDE_Reset_Delay ;20 for 10MHz Z80

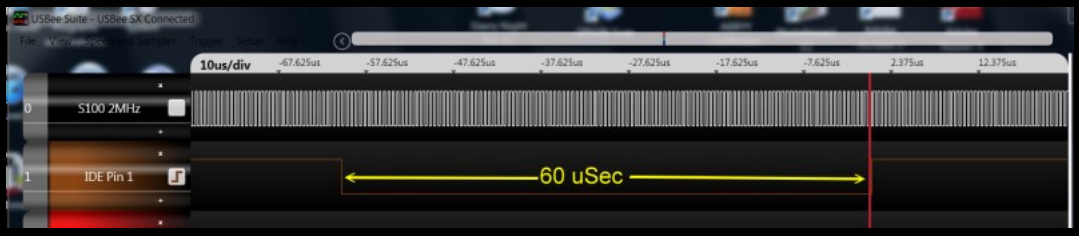
ResetDelay:
        DEC C
        JP      NZ,ResetDelay     ;Delay (reset pulse width)
        XOR     A
        OUT     (IDECport),A      ;No IDE control lines asserted (just bit 7 of port C)

        CALL    DELAY_15          ;Need to wait about 15ms before status is valid
       ……..

DELAY_15:                         ;DELAY ~15 MS
        LD      A,40
```

```
DELAY1: LD      B,0
M0:     DJNZ    M0
        DEC     A
        JR      NZ,DELAY1
        RET
```

Here is a picture of the signal:-



Source Of IDE Adaptors.

Fortunately these single card IDE to CF cards are quite common. They typically cost ~$10.
I have been using the SyBA "SD-CF-IDE-DI IDE" adaptor (NewEgg Item# N82E16822998003), shown here:-



But there are a number of other types. For those that may be interested I have tried some dual CF card adaptors (to allow two drives on one adaptor). So far I have not had success with these "dual" CF card adaptors.

Notes on CHS and LBA Addressing.

On a PC (at least the early ones), sectors are defined is terms of Cylinders, Heads & Sectors (CHS format). Parameters in early MS-DOS versions were passed via software interrupt INT 13H for sector reads and writes. INT 13H designates sectors in a somewhat convoluted way (for historical reasons), parameters passed are always as follows:-

```
AH = 02h
AL = number of sectors to read/written (must be nonzero)
CH = low eight bits of cylinder number
CL = sector number 1-63 (bits 0-5)
high two bits of cylinder (bits 6-7, hard disk only)
DH = head number
DL = drive number (bit 7 set for hard disk)
ES:BX -> data buffer
```

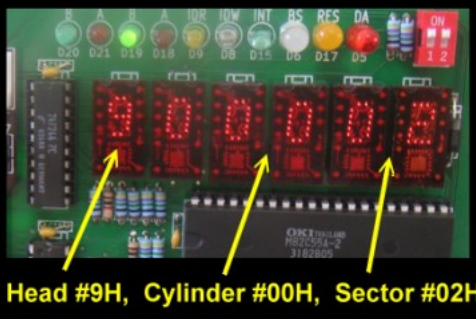
Because only 16 heads are allowed with this format, a hard disk can only get to 512MG in size. This is not a problem for 8 bit CPM systems since disk capacity was always less than 10 MG. To go to higher disk capacities with later version of MSDOS, the PC BIOS's utilized the so called "LBA" sector addressing mode. In this case sectors are numbered on the disk as just one long number. Starting at 0 and working upwards. There are no tracks or heads. Both formats can be implemented on our IDE board.

Programming the LED HEX displays is a little different however for each case. First the drive must be told what format sectors will be addressed in. This is done by sending a key bit to the IDE Drive "Head Register". If bit 6 of this register is set to 0 then the drive expects all information to arrive in LBA mode. If bit 6 is a 1, then the drive expects all sector information to arrive in CHS format.

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit2	Bit 1	Bit 0
Always 1	LBA/CHS mode	Always 1	M/Slave	Heads 0-15			

In the case of LBA addressing, the right most pair of HEX displays are addressed as "Sectors" (0-FFH). The middle pair of displays are addressed as "Low Cylinders" (0-FFH) and the left pair of HEX displays are addressed as "High Cylinders" (0-FFH). This allows up to FFFFFFH (~16.7 Million) sectors to be identified. BTW, for CPM you need only be concerned with the lower 4 HEX displays, outputting to the IDE drive "Sector" & "Low Cylinder" values. In any event, either 2 or 3 bytes are sent to the appropriate IDE "registers" via the 8255 port A.



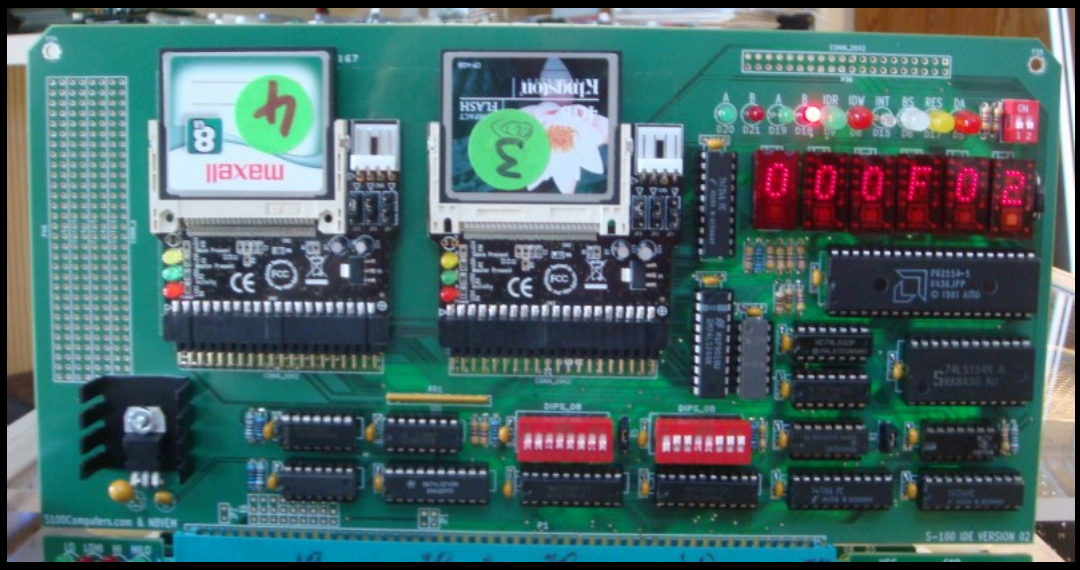


For CHS addressing we do things the same way, however in this case we need display distinguish the Head register on the "High Cylinder" HEX display. The board has a separate access for this display via U12 (74LS244). We trick the display by first sending head information to the IDE drive via the 8255 port A in the normal way. We then get the head information to the upper two HEX displays by sending the data to port B of the 8255 and pulsing the IDE High Cylinder line again. The IDE drive ignores this data on its upper 8 bits data line but the HEX LED displays latch the data. (To illustrate the selected head better I actually put the 4 head bits in the displays high nibble). This may sound more complicate than it is. Please review the 8086 Monitor code (DOS_WR_LBA:) for an example.

If you are sending LBA or true CHS information to a drive for single sector read/writes (e.g. for CPM) it actually does not matter which format the drive uses/expects. In the CHS format there would just be "holes" in the LBA format array. Where all this becomes critical is when we go to MSDOS where as sector read/write command may require a block of up to 80H contiguous sectors to be processed. In this case the disk cannot have sector "holes". When one track of sectors is read the next sector comes from the next head up. When all heads are used only then do we go to the next track. In any BIOS its critical that this process is toughly tested out. The software must religiously either follow this process or via a formula translate a LBA address number into a CHS number. Again please see my 8086 Monitor code (DOS_WR_LBA:) for an illustration of how this can be done.

A Final Dual IDE Prototype Board

Utilizing all the observations above I constructed one more prototype board to make sure everything is correct for what we expect will be a popular S-100 board. Here is a picture of that board:-

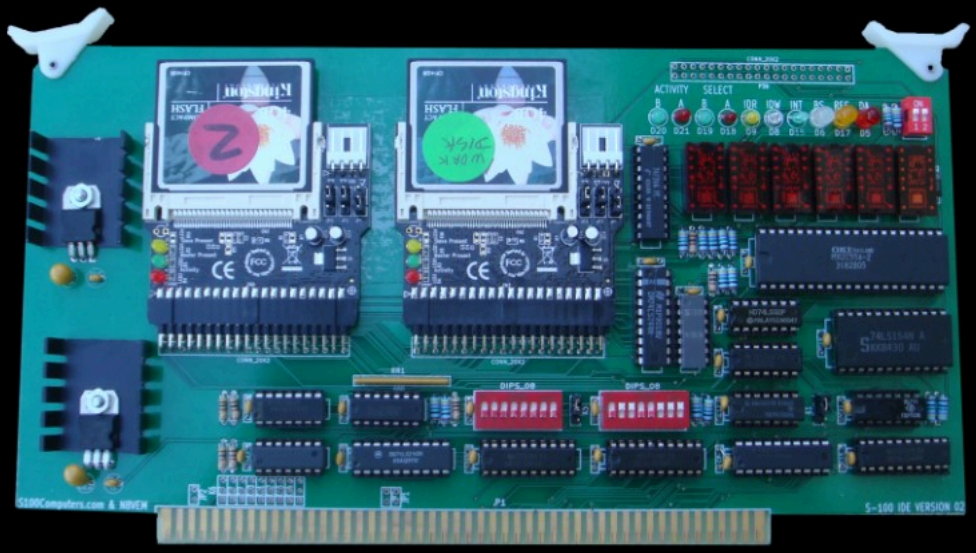


Added where more LED's to let you know which CF card/drive is selected and when a drive is active (RD/WR). The board works fine with 7403's rather than the less common 74F03's. It also works fine with the low power CMOS 82C55's or the older plain NMOS 8255's.

It is interesting to go back and see how this board has evolved. It now has become a fairly sophisticated board and very useful. These days I store everything on CF

cards!

Please note many older S100 bus CPU boards do not set the S-100 bus upper 8 bits to zero for I/O port addressing. In this case jumper K5 2-3, otherwise the CPU will not "see" the ports. (All S100Computers boards use the upper 8 address lines for I/O port decoding). For I/O port address 00xxH, you need to close SW1 so the upper address lines will equate to low).



Dual CF-Card/Hard Disk IDE S-100 Bus Board

Bringing up CPM3 on this IDE Interface S100 Board

In order for this board to be of any practical use you need to install a disk operating system line CPM or MSDOS on the CF cards. This requires a thorough understand of how these disk operating systems function. Please see the Software section of this web site (**BRINGING UP CPM3 FOR THE FIRST TIME ON A IDE HARD DISK BASED SYSTEM**) for further information.

Source of chips

Most of the IC's use on this board are common 74LSxx chips available from numerous sources such a Jameco, Mouser & DigiKey.

Bugs

There seems to be a problem with the drive select A & B timing with some CPU boards, the rising edge signal from U15A pin 1 (WR*) is coming up before the data line D0 has really settled into pin 2 of U16A. You can delay the rising edge signal to U16A, pin 3, by inverting the signal through the spare U15B gate. Bend out pin 5 & 6 of U15 and pin 3 of U16A. Jumper U15A pin 1 to U15 pins 5 & 6 (front of board). Connect U15 pin 4 to U16 pin 3 (front of board). If you experience difficulty switching drives with the MYIDE program software, please consider this patch. (Thanks to Dave Mehaffey & Thomas Owen for pointing this out).

A Version 3 of the S-100 IDE Board.

While the above V2 board seems to work fine for most people, there may be some things "not quite right" with the design for high bus speeds around 10MHz and with fast CPU's like the 80386 and with multi-sector reads with MSDOS etc. I decided the time was right to fine tune the board for another run, a V3 version. There are a few things I wanted to do:-

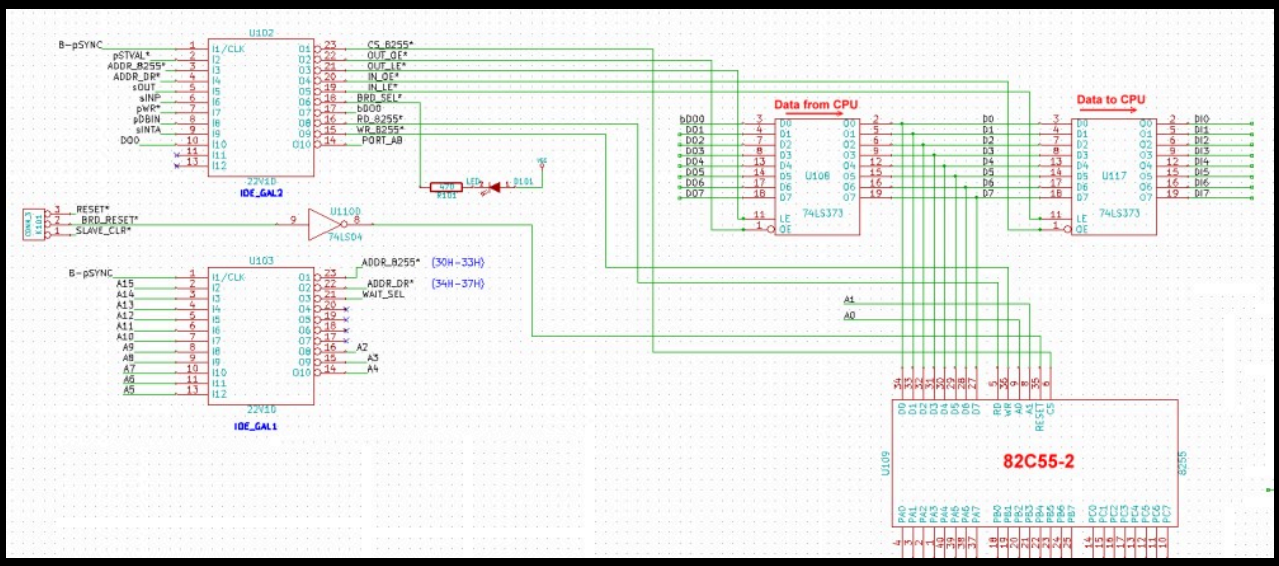
1. I did not want to change the basic 8255 driven circuit and all the software I and others have over the years written for the board. Sure if I were to do it all over again I would probably have done it different using perhaps a faster Zilog PIO or an onboard fast fully dedicated Z80 (as for our ZFDC board) or Propeller etc. The NEC or OKI 82C55-2's are dirt common and seem to be able to handle anything the CPU sends to them.
2. I want to hand lay down broad power traces to all the boards IC's for more even power distribution – particularly to the power hungry HEX displays.
3. I have inserted a trace "Keep out Area" on the front of the board so there is no danger of the IDE adaptors touching a critical trace.
4. I have now switched to a "multi regular" voltage regulator footprint. This allows you to use either:- a 1.5Amp L7805CV, a TO-3 LM323K (3Amps), a Pololu 5 Volt , 2.5 Amp D24V25, or a EzSBC.com 3A regulator called PSU5.
5. I did bend the *no new changes rules* a little and added two 22V10 GALs. This greatly simplifies the board and really speeds up the port addressing and the potential drive select/reset issues some were having. I realize not everybody is familiar with GAL's. Please see here for more information about GALs. The PALASM code is shown below. Again these GAL's are fairly common (Jameco #39159 for the 15ns variety).
6. Correct the potential drive A: or B: switch circuit problem described above.
7. I have added a wait state circuit (0 - 8 I/O wait states) to accommodate very fast S100 boards such as our 80386 and 80486 boards without requiring further generalized I/O wait states on the CPU board for other system I/O ports.

8. People should be able to simply switch IC's from the old board to this new one. Only two new GAL ICs and two 74LS373 are required.

9. Last but least I relabeled much of the Silk Screen to be more relevant. For example placing IC numbers above their pin locations etc.

The main difference with this board is the use of the two 22V10 GALs for port addressing and latching the data into two 74LS373's. At high CPU speeds, and with for example our 68K CPU board, the data written to the 82C55 does not stay long enough for that chip too receive it. The 8255 is an old chip designed originally for old 2-4 MHz CPU's. Any way we can leave the data on the boards bidirectional data bus longer will really help its access. It will also help if we can setup and stabilize the address lines earlier before the CPU read/write signals.

Using GAL's allows us a lot more flexibility in tailoring signals correctly. Here is the relevant circuit:-



First lets look at GAL1. The main function of this GAL is to decode the boards 16 bit port address range. I and most others have, used ports 30-33H to access the 8255. ADDR_8255@ goes low to activate that chips CS* pin 6. Of course any port can be used by programming the GAL differently. Note the ADDR_8255 will go low for inputs AND outputs to the above ports.

We also need one more separate and independent port to select one of the two Drive IDE drives on the board. While a bit wasteful here, (but to mention board compatibility) we use any one of 34H to 37H range of ports. ADDR_DR with data bit 0 high or low, selects Drive A or B using the flip flop U112. (Note I added the pSYNC input for later register use "just in case", its not currently used). The simple PALASM code used is shown below:-

```
PIN 1 bpSYNC                        ;S100 bus address valid when high (Not used)
PIN 2 A15                           ;S100 Bus Address line 15
PIN 3 A14                           ;S100 Bus Address line 14
PIN 4 A13                           ;S100 Bus Address line 13
PIN 5 A12                           ;S100 Bus Address line 12
PIN 6 A11                           ;S100 Bus Address line 11
PIN 7 A10                           ;S100 Bus Address line 10
PIN 8 A9                            ;S100 Bus Address line 9
PIN 9 A8                            ;S100 Bus Address line 8
PIN 10 A7                           ;S100 Bus Address line 7
pin 11 A6                           ;S100 Bus Address line 6

PIN 13 A5                           ;S100 Bus Address line 5
Pin 14 A4                           ;S100 Bus Address line 4
Pin 15 A3                           ;S100 Bus Address line 3
Pin 16 A2                           ;S100 Bus Address line 2
Pin 17 NC
Pin 18 NC
Pin 19 NC
Pin 21 WAIT_SEL
Pin 22 /ADDR_DR                     ;Drive Switch port (low)
Pin 23 /ADDR_8255                   ;8255 Select, (I/O Input or output) (low)


;---------------------------------- Boolean Equation Segment ------
```

```
EQUATIONS

ADDR_8255     =  /A15 * /A14 * /A13 * /A12 * /A11 * /A10 * /A9 * /A8        ;Ports 30H - 33H (OUT/IN to ports)
                * /A7 * /A6 * A5 * A4 * /A3 * /A2                           ;001100xx


ADDR_DR       =  /A15 * /A14 * /A13 * /A12 * /A11 * /A10 * /A9 * /A8        ;Ports 34H - 37H (Pulse OUT only)
                * /A7 * /A6 * A5 * A4 * /A3 * A2                            ;001101xx

WAIT_SEL =     ADDR_DR + ADDR_8255                                          ;For I/O wait states
```

GAL2 is a bit more complicated. Lets first consider data being written to the 8255. We want the data available as long as possible to the chip, so early on we bring its OE* (pin 1) low as soon as a valid port address is available and either the associated S100 bus sOUT or sINP status signals are presented. The latter come up before the CPU's actual read or write pulse -- at least for the older S100 bus CPU boards. The 74LS373 (U108) will see and present its input data on its output pins when its LE (pin11) goes HIGH. It will latch this data on the falling edge of LE. We will use the inverted CPU pWR* for this purpose. As long as sOUT or sINP remains high afterwards the 8255 will still be able to pick up the data. This will vary from CPU board to board, but it should never be shorter than the current bus master CPU's actual pWR* signal length (+ wait states).

The approach is essentially the same for reading data from the 8255. This is generally not a problem with fast CPU's, but just to be on the safe side I'll add a 74LS373 (U117) there as well.

The 8255 requires its own WR (pin 36) and RD (pin 5) signals. These are active low, not a problem for the S100 bus pWR*, but the s100 bus active high pDBIN signal needs to be inverted coming out of the GAL (pin16) for the 8255.

Finally the 8255's own CS* needs to be provided. This comes from pin 23 of GAL2.

I have added as inputs to GAL2 the S100 signals pSYNC, pSTVAL and sINTA in case at some future date it is necessary to clock /register the GAL outputs or if for some strange reason interrupt acknowledge responses are interfering with the boards function. Here is the simple PALASM code for GAL2.

```
PIN 1 bpSYNC                                ;S100 bus address valid when high (Not used)
PIN 2 /bsVAL                                ;S100 bus address valid when low (Not used)
PIN 3 /ADDR_8255                            ;Select 8255 address range (from GAL1)
PIN 4 /ADDR_DR                              ;Drive Switch port address range (from GAL1)
pin 5 sOUT                                  ;S100 bus signal
PIN 6 sINP                                  ;S100 bus signal
PIN 7 /pWR                                  ;S100 bus signal
PIN 8 pDBIN                                 ;S100 bus signal
PIN 9 sINTA                                 ;S100 bus signal
PIN 10 DO0                                  ;S100 bus data line 0 output
PIN 11 NC

PIN 13 NC
Pin 14 PORT_AB
Pin 15 WR_8255                              ;Read signal to 8255 (low)
Pin 16 RD_8255                              ;Read signal to 8255 (low)
Pin 17 bDO0                                 ;Buffered bus data line 0 output
Pin 18 /BRD_SEL                             ;Board Select LED (low)
Pin 19 /IN_LE                               ;LE for 74LS373, U117
Pin 20 /IN_OE                               ;OE for 74LS373, U117
Pin 21 /OUT_LE                              ;LE for 74LS373, U108
Pin 22 /OUT_OE                              ;OE for 74LS373, U108
Pin 23 /CS_8255                             ;CS for 8255

;---------------------------------- Boolean Equation Segment ------

EQUATIONS

IN_LE    =    ADDR_8255 * sINP * /pDBIN             ;Latch data on Low to High

IN_OE    =    ADDR_8255 * sINP * /sINTA             ;Enable input buffer
```

```
OUT_LE   =    ADDR_8255 * sOUT * pWR               ;Latch data on Low to High

OUT_OE   =    ADDR_8255 * sOUT * /sINTA             ;Enable input buffer

CS_8255  =    ADDR_8255 * sINP * /sINTA             ;8255 CS (Read & Write)
           + ADDR_8255 * sOUT * /sINTA

PORT_AB  =    ADDR_DR * sOUT  * pWR                 ;Write only

BRD_SEL  =    CS_8255
            + ADDR_DR * pWR@

bDO0      =   DO0
RD_8255   =   /pDBIN
WR_8255   =   pWR
```

There are four possible options for the 5V Voltage regulators:--
You can use a 1.5 Amp L7805CV TO-220 type regulator with a heat sink
You can use a traditional LM323K 3A TO-3 regulator with a heat sink.
You can use the Pololu 5 Volt , 2.5 Amp D24V25 Switching regulator (See here).
Alternatively you can use these newer direct TO-3 footprint switching regulator by EzSBC.com called PSU5.

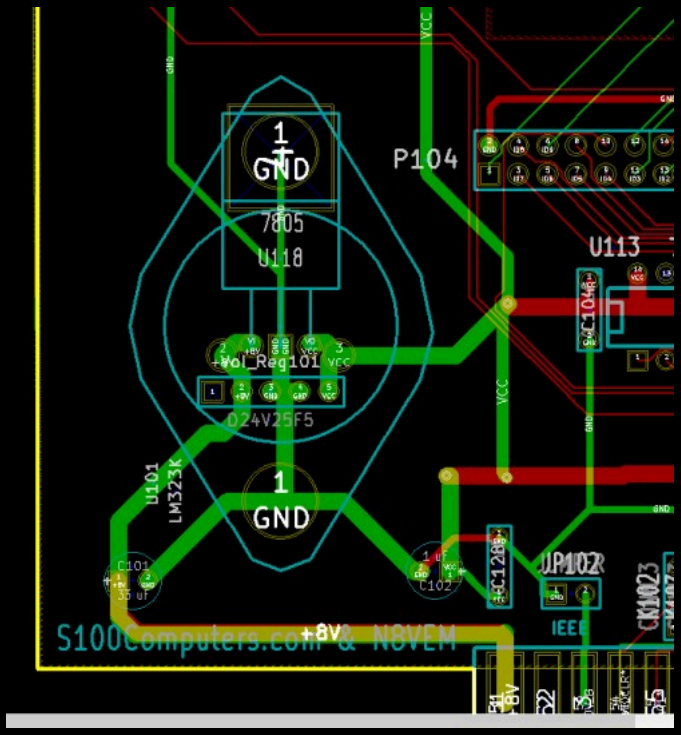
Both the Pololu and EzSBC units run cold, no heat sink is required. I have use the PSU5 for the recent SMB V3 board.



Pololu 5V, 2.5A Step-Down Voltage Regulator D24V25F5.



The footprint on the board allows any one of these voltage regulator to be used:-



Here is a picture of a the final production V3 board. Thanks goes to Dave Mehaffey (Yoda), David Fry, Ian May and others in arriving at this schematic/board layout.

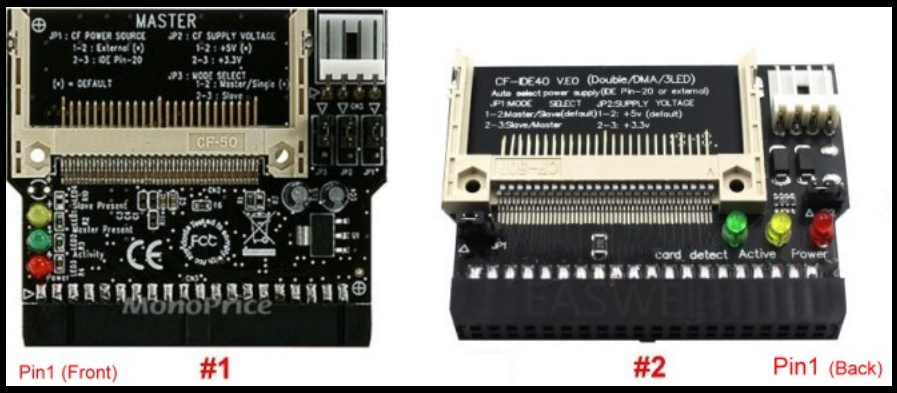


IDE/CF Card V3 S100 Board

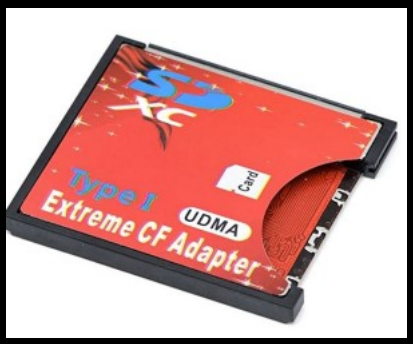
The board seems to be working fine with both CPM3 and MSDOS V4.01 with all our CPU's.

Since the original IDE/CF Card board came out (in 2004) SYBA Multimedia has come out with a smaller version of the adaptor. It behaves exactly the same BUT the IDE connector orientation has been flipped around. As you look at the front of the adaptor pin 1 is at the back top row on the right (#2 below). This adaptor will not work with this board. You need the "old" adaptors (#1 below).

You can "kludge" it by putting the new adaptor in with its front surface (LEDs etc.) facing the board. Not a great solution because its hard to insert/remove the CF cards with this configuration.



To utilize SD cards in early designs of this board I utilized SPI SD card adaptors such as the Adfruit or Sparkfun Adaptors. While they worked fine, actual card access was 10X slower that for CF Cards. This is due to the single bit SPI public domain interface rather than the 4 bit licensed interface. Fortunately again there are IDE to SD card adaptors that completely take care of the complex process of SD card initialization and access at CF card speeds. Here is a picture of the one I currently use.

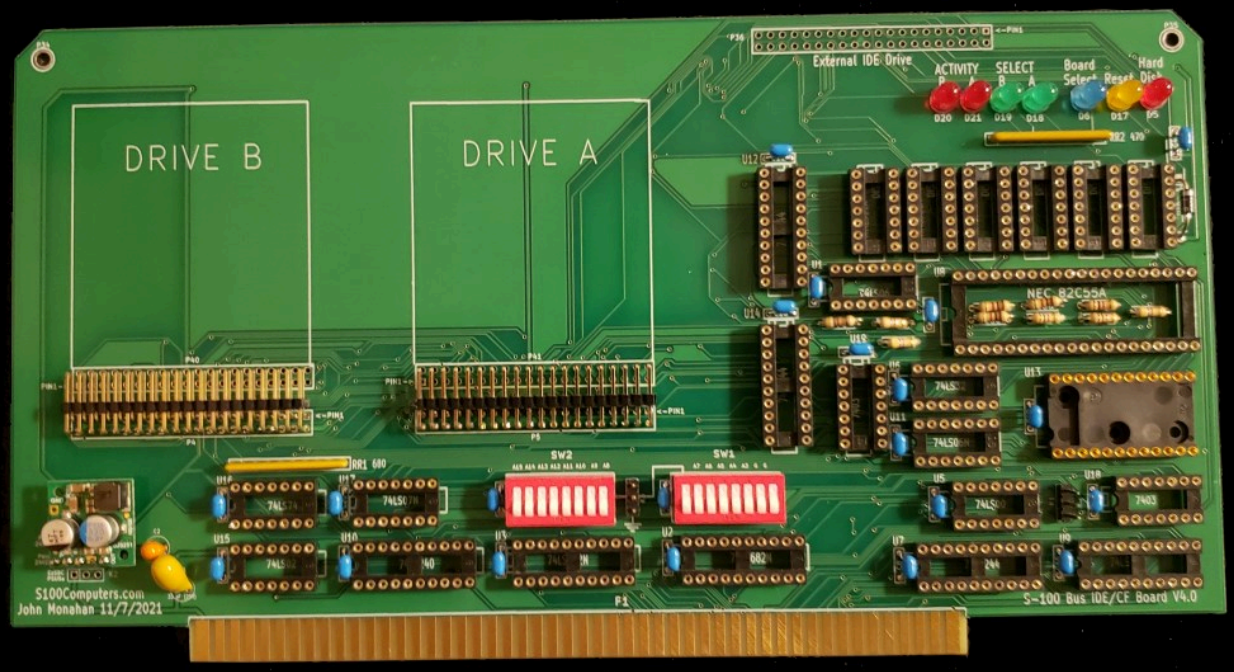


There are also mini to micro SD card adaptors that fit into these units. So our IDE sockets can accommodate either:-
A Ribbon cable connected "real" IDE hard disk.
A CF Card
A Mini SD Card
or a Micro SD Card.

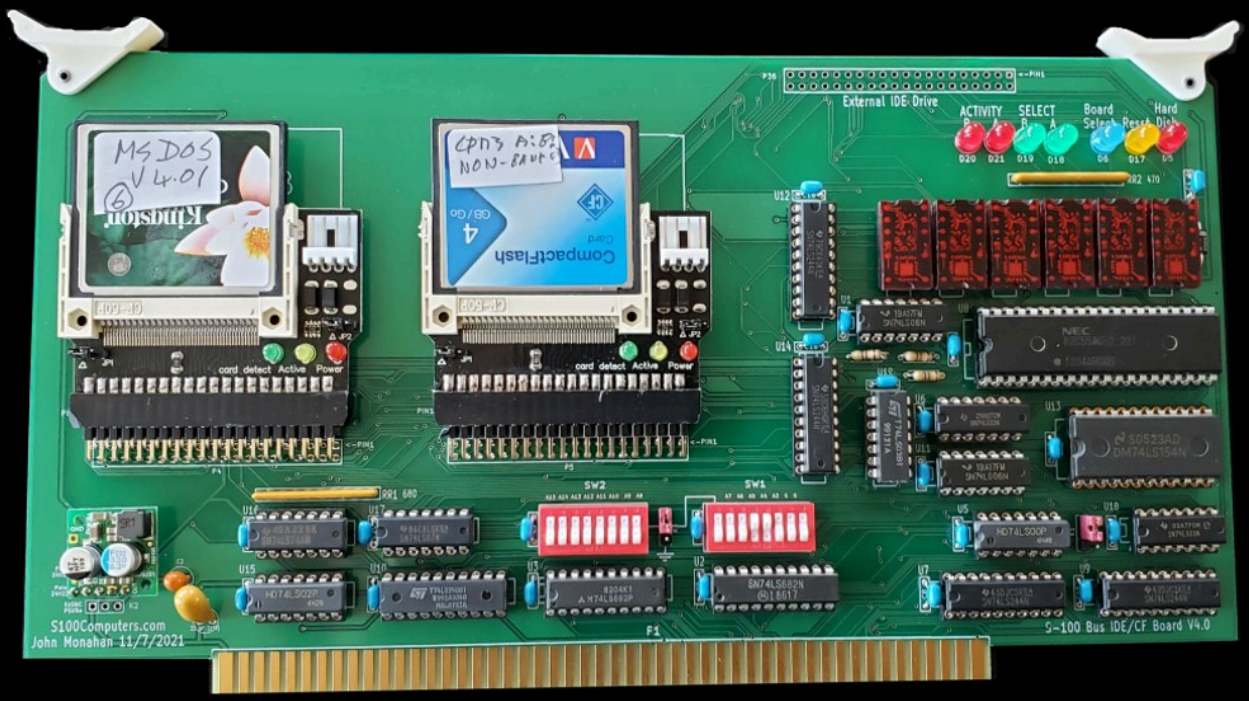
As far as the software is concerned (see below) all behave the same.

A Version 4V of the S-100 IDE Board.

While the above V3 board seems to work fine for most people, having to obtain GAL's and a programmer to program them is a problem for some people.
Also the SYBA Multimedia IDE-CF card adaptor #1 shown above is getting difficult to find.
I decide to do another version of this very popular board. (Over 300 have gone out over the years). This version goes back to using simple 74LSXX chips and requires no GALs or programming.
Also sockets for both the #1 and #2 IDE/CF Card adaptors are onboard. While not absolutely essential, try and get NEC 82C55A's.
Here is a picture of the board before adding IC's.



You can use either Pololu or EzSBC 5V regulators. There are 2 pairs of socket holes for both "Drives". The newer SYBA Multimedia IDE-CF card adaptor #2 goes in the lower sockets.
Try and keep the headers raised as much as you can from the board. The right-angled short pins should just barely show for soldering at the back of the board.
There are only two jumpers on the board.
K2 defines if you are using 8 or 16 bit I/O port addressing. Jumper 1-2 if not. If you are (e.g. with MSDOS/8086) jumper 2-3 and (normally) use a high byte of 00H.
K1 defines if the S100 Bus signals "RESET" or "SLAVE CLEAR" resets the board, (the 74LS74 flip-flop to drive A).
Here is a picture of the finished board.



V4 Dual CF-Card/Hard Disk IDE Board

CPM3 IDE/CF Card Download Images.

In order to quickly get people up and running with CPM3 with our S10Computers boards here are some CF Card images which when loaded on to a CF card should allow you to boot up a complete CPM3 system immediately.

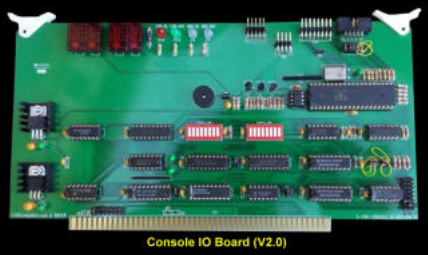
Before doing so it might be a good idea to read up on the boot process for CPM3 first. Please see here.

I will provide more "CF card images" over time.

Image 1. Drives A:+B:

This is a minimal CPM3 Disk with two CF cards on Drive A: and B: It requires the Propeller Console IO Board for I/O and a 64K RAM board with RAM from 0H-EFFFH uninterrupted by ROM's etc.
Typing the "P" command in the Z80 Master monitor (at F000H-FFFFH in ROM) is all you need to boot CPM3.
If you don't have that monitor you need to read into RAM, starting at 100H, the first 12 sectors on track 0 of the CF card. Then jump to 100H in RAM to launch CPM3.
Here is a picture of the setup boards:-

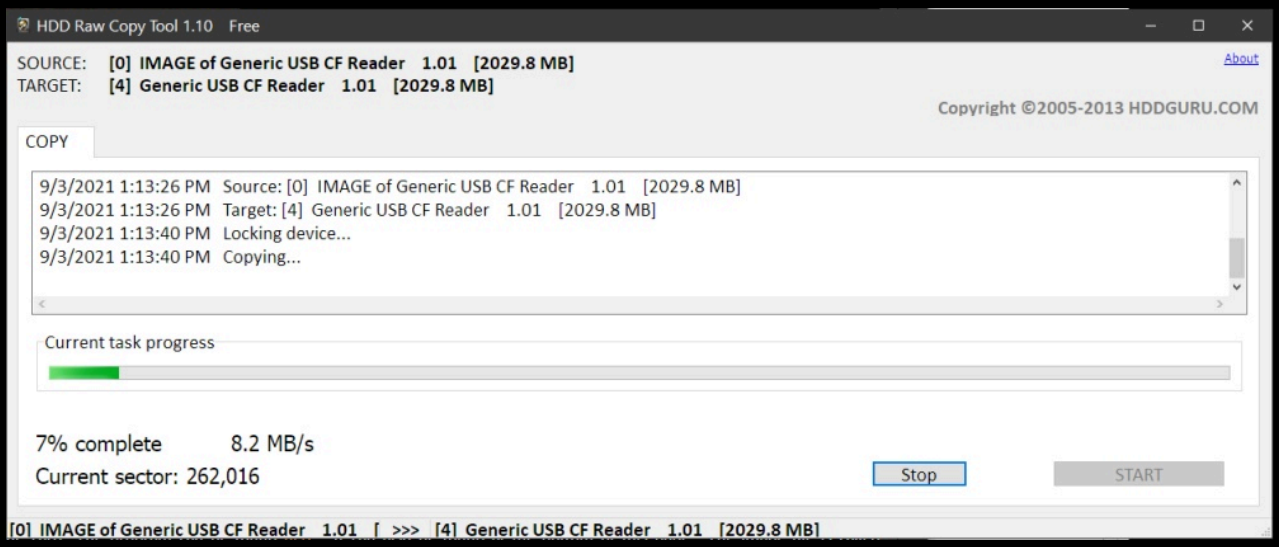








Download the HHD Raw Copy program (described here) from the bottom of that page.
Download the FPGA_DC_CPM_Image2D.imgc file from the same page and use HHD Raw Copy to unpack the FPGA_DC_CPM_Image0.imgc on to a CF card (of at least 4GB).
Actually since our CPM3 disk is only 0FFH tracks, 0FFH Sectors maximum, you can probably stop the unpacking process after sector 0FFFFH (65,535). Here is how the "unpacking" dialog looks:-



Then simply insert the CF card into any one of the above Disk Controller boards. You should be able to boot CPM directly with the monitor P command.

```
CP/M V3.0 Loader
Copyright (C) 1982, Digital Research

 BIOS3    SPR  BD00  0900
 BDOS3    SPR  9C00  2100

 39K TPA


64K CP/M VERSION (Build 0.4) (John Monahan 10/13/2021)
A: & B: = IDE CF cards (61 Sec/Track)
A>dd
Name    Ext Bytes   Name    Ext Bytes   Name    Ext Bytes   Name    Ext Bytes
ASM86   COM   28K | DIR     COM   16K | PCLOAD  COM    2K | SPM3    SYS   12K
CCP     COM    4K | DU      COM    8K | PCPUT   COM    2K | SWEEP   COM   28K
CLEANUP COM    2K | EDIT    COM   14K | PIP     COM    8K | USBGET  COM    2K
COMPARE COM    2K | FDC$DIAGCOM  12K | PIP22   COM    8K | UDSETCRTCOM    8K
CPM-LINKCOM   16K | FIND    COM    2K | PROFILE SUB    2K | UEDIT   INI    2K
CPM3    LIB    4K | FLUSH   COM    2K | PROM    COM    8K | UF      COM   16K
CPM3    SYS   12K | GET     COM    8K | PUT     COM    8K | XLT00   OVL    8K
CPM3$2D SYS   12K | HEXCOM  COM    2K | QDU     COM    4K | XLT01   OVL   10K
CPM3$6D SYS   14K | HSYSGEN COM   10K | RENAME  COM    4K | Z80     LIB    8K
CPM3-OK SYS   24K | IDE$DIAGCOM   12K | RESET   COM    2K | ZASM    COM   12K
CRCK    COM    2K | INITDIR COM   32K | RMAC    COM   14K | ZASM-TDLCOM   14K
CRT     TBL    8K | LINK    COM    6K | SAVE    COM    2K | ZASMB   COM   14K
DATE    COM    4K | LOAD    COM    4K | SET     COM   12K | ZFDCDIAGCOM   22K
DD      COM    4K | MAC     COM   12K | SETDEF  COM    4K | ZSID    COM   10K
DDD     COM   16K | MACROS  LIB    2K | SETIO   COM    2K
DDT     COM    6K | MYIDE   COM   14K | SHOW    COM   10K
DEVICE  COM    8K | PCGET   COM    2K | SPLIT   COM    2K
65 File(s), occupying 584K of 7744K total capacity
486 directory entries and 4228K bytes remain on A:
A>_
```

Note this is a simple non-banked version of CPM3.

BUGS

There is one small error on the board (picked up by Rick Bromagem, thanks Rick). The 1K pull-up resistor R118 to pin 10 of U106 (a 74LS01), is absent. To correct, on the back of the board connect a 1K resistor between pin 10 and 14 of U106.

A second error has shown up. When using a very fast CPU (such as our 80386 board with Phi at 10MHz) , the V3 IDE board does not initialize correctly, (e.g. "N" command with the 80386 monitor command). This appears to be due to the early arrival of the pDBIN and/or pWR* signals on the bus with this CPU board. This can be corrected by delaying the board chip select signals until the S100 bus pSYNC has returned low. The equations in the above IDE_ GAL1 need to be changed to:-

```
ADDR_8255     =  /A15 * /A14 * /A13 * /A12 * /A11 * /A10 * /A9 * /A8        ;Ports 30H - 33H (OUT/IN to ports)
              * /A7 * /A6 * A5 * A4 * /A3 * /A2 * /pSYNC                    ;001100xx


ADDR_DR       =  /A15 * /A14 * /A13 * /A12 * /A11 * /A10 * /A9 * /A8        ;Ports 34H - 37H (Pulse OUT only)
              * /A7 * /A6 * A5 * A4 * /A3 * A2  * /pSYNC                    ;001101xx

WAIT_SEL =     ADDR_DR + ADDR_8255                                          ;For I/O wait states
```

This completely corrects the problem. Note this is not an issue with our Z80, 8088, 8086 or 80286 boards though it would do no harm to use the above code with them as well..

David Fry has spent some time on the CF card compatibility issue with the above IDE Boards where some CF cards will not boot CPM properly using the monitor "P" command. He noted that in the V5.4 monitor (and previous versions), that HBOOTCPM routine performs a 12 sector read in one pass.

```
LD      D,SEC_COUNT         ;Count of CPM sectors we wish to read
LD      E,REGcnt
CALL    IDEwr8D
```

He concluded that some cards require that the block Read requests be in powers of 2 (2,4,8 or 16...). So he rewrote this section of the monitor to perform the CPMLDR read using single sector reads in a loop of 12 iterations. This seems to have solved that annoying problem, allowing many if not all CF cards to work fine. Thanks Dave!

The Z80 monitor code V5.5 below now incorporates this modification and should be updated in the onboard ROMs of all our Z80 CPU boards.

Damian Wildie noticed that the wait state circuit on the IDE/CF board was not functioning. He traced the issue to the GAL1 code which added a not pSYNC term to the address select, ADDR_8255 and ADDR_DR, equations. WAIT_SEL is derived from ADDR_8255 and ADDR_DR and is now only true when pSYNC is false. Since WAIT_SEL and pSYNC are handed by U106A, the load pulse for the wait state shift register U107 is never generated. He changed the WAIT_SEL equation

From: WAIT_SEL = ADDR_DR + ADDR_8255
To: WAIT_SEL = /A15 * /A14 * /A13 * /A12 * /A11 * /A10 * /A9 * /A8 * /A7 * /A6 * A5 * A4 * /A3

It now seems to be working correctly and the LED 112 actually flickers with wait states inserted. BTW, its seldom you will need wait states with this board, particularly if you use a CMOS version of the 8255A. Thanks Damian!

Parts (for the V4 board)

The TIL311 hex displays are optional. They are also expensive... shop around for a deal on eBay if you want them.
U8 should ideally be an NEC82C55AC-2
U18 and U19 are listed as a 7403 in the schematic - these are now obsolete parts. You can substitute them with the 74LS03's.

A Production S-100 Board

Realizing that a number of people might want to utilize a board like this together with a group of people on the Google Groups S100Computers Forum, "group purchases" are made from time to time.

Please see here for more information

The links below will contain the most recent schematic of the above boards.
Note, it may change over time and some IC part or pin numbers may not correlate *exactly* with the text in the article above.

MOST CURRENT IDE BOARD SCHEMATIC (V2a) (V2-corrected, FINAL-2a, 5/10/2014)
MOST CURRENT IDE BOARD LAYOUT (V2a) (V2-corrected, FINAL-2a, 5/10/2014)
Most current KiCAD files for V2 board (S100 IDE-002.zip 11/6/2014)

MOST CURRENT PARTS LIST (V2-C2, FINAL, 6/12/2011)
Another BoM List (Alex Jacobs)

David Fry has done further work with this V2 board and has written up some very useful notes which you should look at if you are using the V2 board.
Please see board build info, CPM installation info and the corresponding .zip files,
Thanks Dave!

A 4K ROM based version of MYIDE.ASM can be see here and downloaded here. (3/15/2011)

ORIGIONAL V1 IDE BOARD SCHEMATIC (V1_0 1/17/2011)
ORIGIONAL V1 IDE BOARD LAYOUT (V1.0 1/17/2011)
ORIGIONAL V2 IDE BOARD SCHEMATIC (V2a 5/10/2015)
ORIGIONAL V2 IDE BOARD LAYOUT (V2a 5/10/2015)

V3a Gerber Zip Files (2/3/2018)
V3a IDE KiCAD Zip Files (2/3/2018)
V3 & V3a IDE Final Schematic (5/16/2015)
V3 & V3a IDE GAL File (Corrected) (5/28/2020)
BOM for V3 & V3a IDE Board (From Rick Bromagan 12/26/2015)

The corresponding files for the V4 board can be found here:-
V4 IDE Board KiCAD Folder (V4.0 11/24/2021
V4 IDE BOARD SCHEMATIC (PDF file) (V4.0 11/24/2021
V4 Gerber Zip Files (.zip file) (V4.0 11/24/2021)
V4 IDE Board BOM File (.csv file) (V4.0 12/10/2025)

The most recent updated version of the MYIDE.COM diagnostic program can be seen here:-

MOST CURRENT MYIDE.ZIP (V4.2) (9/20/2026)

Many CPM3 BIOS'es using this card are described in detail here and here. (9/20/2026))

Other pages describing my S-100 hardware and software.

Please click here to continue...

This page was last modified on 10/18/2026